# Practical No. 6

## [Process Synchronization]

Considered there are N philosophers seated around a circular table with one chopstick between each pair of philosophers. There is one chopstick between each philosopher. A philosopher may eat if he can pick up the two chopsticks adjacent to him. One chopstick may be picked up by any one of its adjacent followers but not both.

Write a program to solve the problem using process synchronization technique.

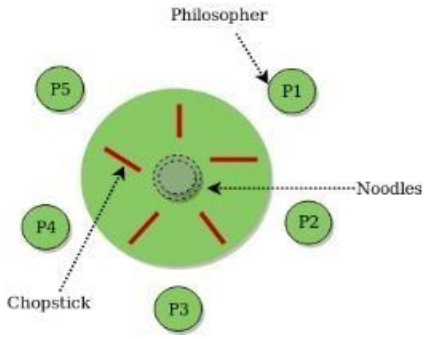


**Fig: 1. Dining Philosopher Problem**

## INPUT :

```
~
  GNU nano 8.7                                                    dining.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define N 5

pthread_t philosopher[N];
sem_t chopstick[N];
sem_t room;  // allows only N-1 philosophers

void* dine(void* num) {
    int id = *(int*)num;

    while (1) {
        printf("Philosopher %d is Thinking\n", id);
        sleep(1);

        sem_wait(&room);  // enter room

        sem_wait(&chopstick[id]);
        printf("Philosopher %d picked left chopstick\n", id);

        sem_wait(&chopstick[(id + 1) % N]);
        printf("Philosopher %d picked right chopstick\n", id);

        printf("Philosopher %d is Eating\n", id);
        sleep(2);

        sem_post(&chopstick[id]);
        sem_post(&chopstick[(id + 1) % N]);

        printf("Philosopher %d finished eating\n", id);

        sem_post(&room);  // leave room
    }
}

int main() {
    int i, id[N];

    sem_init(&room, 0, N-1);

    for (i = 0; i < N; i++)
        sem_init(&chopstick[i], 0, 1);

    for (i = 0; i < N; i++) {
        id[i] = i;
        pthread_create(&philosopher[i], NULL, dine, &id[i]);
    }

    for (i = 0; i < N; i++)
        pthread_join(philosopher[i], NULL);

    return 0;
}
```

**OUTPUT :**

```
~

chanc@Chanchal MSYS ~
$ nano dining.c

chanc@Chanchal MSYS ~
$ gcc dining.c -o dining -lpthread
./dining
Philosopher 0 is Thinking
Philosopher 1 is Thinking
Philosopher 2 is Thinking
Philosopher 3 is Thinking
Philosopher 4 is Thinking
Philosopher 4 picked left chopstick
Philosopher 3 picked left chopstick
Philosopher 0 picked left chopstick
Philosopher 2 picked left chopstick
Philosopher 0 picked right chopstick
Philosopher 0 is Eating
Philosopher 0 finished eating
Philosopher 4 picked right chopstick
Philosopher 0 is Thinking
Philosopher 1 picked left chopstick
Philosopher 4 is Eating
Philosopher 4 finished eating
Philosopher 3 picked right chopstick
Philosopher 4 is Thinking
Philosopher 0 picked left chopstick
Philosopher 3 is Eating
Philosopher 2 picked right chopstick
Philosopher 3 finished eating
Philosopher 2 is Eating
Philosopher 3 is Thinking
Philosopher 4 picked left chopstick
Philosopher 2 finished eating
Philosopher 1 picked right chopstick
Philosopher 2 is Thinking
Philosopher 3 picked left chopstick
Philosopher 1 is Eating
Philosopher 1 finished eating
Philosopher 0 picked right chopstick
Philosopher 1 is Thinking
Philosopher 0 is Eating
Philosopher 2 picked left chopstick
Philosopher 0 finished eating
Philosopher 4 picked right chopstick
Philosopher 0 is Thinking
Philosopher 1 picked left chopstick
Philosopher 4 is Eating
Philosopher 4 finished eating
Philosopher 3 picked right chopstick
Philosopher 4 is Thinking
Philosopher 0 picked left chopstick
Philosopher 3 is Eating
Philosopher 3 finished eating
Philosopher 2 picked right chopstick
Philosopher 2 is Eating
Philosopher 3 is Thinking
Philosopher 4 picked left chopstick
```